PostgreSQL Advanced Features

A DevOps / Backend Engineer Study Guide

Focus: PostgreSQL capabilities that matter beyond basic CRUD, with practical SQL examples and interview-ready comparisons with MySQL.

Core topics	What you will learn
Data types	JSONB, arrays, UUID, enums, ranges, composite types, domains.
Indexes	B-tree, GIN, GiST, SP-GiST, BRIN, expression and partial indexes.
Constraints	Primary/foreign keys, CHECK, UNIQUE, exclusion constraints.
Performance	Materialized views, query design, indexing choices.
Security	Roles, privileges, row-level security, least privilege.
Extensibility	Extensions, custom types/functions, foreign data wrappers.
Geospatial	PostGIS concepts and how they map to driver-location workloads.

Important: MySQL also supports many modern features, including JSON, CTEs, window functions and spatial functionality. This guide focuses on areas where PostgreSQL is especially rich or distinctive rather than claiming that MySQL lacks modern capabilities.

1. PostgreSQL vs MySQL: the right mental model

Both are mature relational database management systems. PostgreSQL is often chosen when an application benefits from advanced SQL, a rich type system, extensibility, sophisticated indexing, complex constraints, strong database-side security policies, or geospatial workloads. MySQL is also capable and is widely used for conventional transactional and web workloads.

Area	PostgreSQL	MySQL
Relational SQL	Strong SQL and advanced query capabilities	Strong SQL and advanced query capabilities
JSON	JSON and JSONB; strong indexing/operator support for JSONB	Native JSON type and JSON functions/indexing
Advanced index methods	B-tree, GIN, GiST, SP-GiST, BRIN, etc.	B-tree, hash and other index capabilities depending on engine/features
Partial indexes	Native partial-index support	No direct PostgreSQL-style partial index syntax
Range types	Native range/multirange types	No equivalent native range type
Exclusion constraints	Native table constraint for preventing conflicting combinations, often used with ranges	No direct equivalent
Materialized views	Native materialized views	No native equivalent in the same sense
Row-level security	Native RLS policies	Different security mechanisms; no direct equivalent to PostgreSQL RLS
Extensibility	Rich extension/type/operator framework	Plugin/component ecosystem with a different model

Interview framing

Do not answer “PostgreSQL is faster” or “PostgreSQL is better.” A stronger answer is workload-oriented: the choice depends on the data model, query complexity, ecosystem, operational requirements, team skills, and required features.

2. Rich data types and JSONB

PostgreSQL provides a broad type system. Useful advanced types include JSON/JSONB, arrays, UUID, enums, range and multirange types, network addresses, geometric types, composite types and domains. This lets you model application data more directly.

JSONB

JSONB stores JSON in a binary representation optimized for processing. PostgreSQL provides operators and functions for querying JSONB and supports GIN indexing for many JSONB access patterns.

```
CREATE TABLE vehicles (
    id BIGSERIAL PRIMARY KEY,
    details JSONB
);

INSERT INTO vehicles (details) VALUES
('{"make": "Toyota", "model": "Innova"}');

SELECT *
FROM vehicles
WHERE details->>'make' = 'Toyota';
```

Arrays

```
CREATE TABLE driver_profiles (
    id BIGSERIAL PRIMARY KEY,
    supported_vehicle_types TEXT[]
);

INSERT INTO driver_profiles (supported_vehicle_types)
VALUES (ARRAY['Auto','Sedan','SUV']);
```

UUID

UUIDs are useful when identifiers need to be hard to guess and when identifiers may be generated independently across services. PostgreSQL supports UUID as a native type.

Range types

```
CREATE TABLE availability (
    driver_id BIGINT,
    available_window tstzrange
);

-- Example value: [2026-09-24 09:00, 2026-09-24 18:00)
```

3. Advanced indexing

Indexes are one of PostgreSQL's most important advanced areas. The right index depends on the operator and query pattern, not simply on the existence of a WHERE clause.

Index type / capability	Typical use
B-tree	Equality, ordering, range comparisons; common default index
GIN	Inverted indexing for JSONB, arrays and text-search-related structures
GiST	Extensible indexing; common for ranges and spatial workloads
SP-GiST	Specialized partitioned search structures for supported operator classes
BRIN	Very large tables where physical row order correlates with the indexed value
Expression index	Queries on computed expressions such as LOWER(email)
Partial index	Index only rows matching a predicate such as status = active
Covering / INCLUDE	Store extra columns in an index to support index-only plans in suitable cases

Expression index example

```
CREATE INDEX idx_users_lower_email
ON users (LOWER(email));

SELECT *
FROM users
WHERE LOWER(email) = 'test@example.com';
```

Partial index example

```
CREATE INDEX idx_active_bookings
ON bookings (driver_id)
WHERE status = 'active';
```

Senior-level point

An index is not automatically beneficial. It adds write and storage overhead and may be ignored by the planner when the selectivity or cost model says a sequential scan is cheaper. Always validate with EXPLAIN / EXPLAIN ANALYZE against realistic data.

4. Exclusion constraints and range logic

PostgreSQL range types become especially powerful when combined with exclusion constraints. An exclusion constraint can prevent combinations of rows that would conflict according to defined operators.

```
CREATE TABLE room_booking (
    room_id BIGINT,
    booking_period tstzrange,
    EXCLUDE USING gist (
        room_id WITH =,
        booking_period WITH &&
    )
);
```

The && operator means “overlaps”. The constraint says: for the same room, another row must not have an overlapping booking period. This is a database-enforced rule, not merely an application convention.

Jhanvi Cars example

A similar design can be useful for scheduling or reservation windows where overlapping allocations must be prevented. Whether you use it for driver shifts, vehicle reservations, or another feature depends on the exact business rule.

Why this matters

- The rule is enforced centrally in the database.
- Concurrent requests cannot simply bypass the rule by racing through application checks.
- The design can make complex temporal constraints explicit and auditable.

5. Materialized views

A normal view stores a query definition. A materialized view stores the result of a query and can be refreshed later. This is useful when a query is expensive but its result does not need to be recomputed for every dashboard request.

```
CREATE MATERIALIZED VIEW daily_revenue AS
SELECT
    DATE(created_at) AS day,
    SUM(fare) AS revenue
FROM bookings
GROUP BY DATE(created_at);

REFRESH MATERIALIZED VIEW daily_revenue;
```

Admin portal use case

Suppose the Admin portal needs daily ride counts and revenue over millions of bookings. Instead of recomputing the aggregation on every page load, a materialized view can be refreshed on a schedule appropriate to the business need.

Trade-off

- Faster reads for expensive aggregations after refresh.
- Results can become stale between refreshes.
- Refresh strategy and indexing need to be designed around the reporting SLA.

6. Row-Level Security (RLS)

Row-Level Security lets PostgreSQL apply policies that control which rows a database role can access or modify. It is useful for multi-tenant or data-isolation requirements.

```
ALTER TABLE bookings ENABLE ROW LEVEL SECURITY;

CREATE POLICY customer_booking_policy
ON bookings
FOR SELECT
USING (customer_id = current_setting('app.user_id')::bigint);
```

The example shows the concept only. In a production system, the exact policy design, session-variable handling, role model, and connection-pool behavior must be carefully designed and tested.

Why DevOps / platform engineers should care

- Security can be enforced closer to the data rather than relying entirely on application code.
- Policies can reduce the blast radius of a compromised application role.
- RLS introduces operational considerations around role management, connection pooling and migrations.

7. Extensions and Foreign Data Wrappers

PostgreSQL is designed to be extensible. Extensions can add functions, types, operators, indexes, and other database objects. This is one of the reasons PostgreSQL is used as a platform for specialized workloads.

Extensions

```
CREATE EXTENSION IF NOT EXISTS postgis;
```

PostGIS is the best-known example. It adds rich geospatial data types, functions and indexes for location-aware systems.

Foreign Data Wrappers (FDW)

Foreign Data Wrappers allow PostgreSQL to access external data sources through foreign tables. PostgreSQL includes postgres_fdw for connecting to another PostgreSQL server.

```
CREATE EXTENSION postgres_fdw;

-- The exact SERVER / USER MAPPING / FOREIGN TABLE
-- configuration depends on the remote PostgreSQL system.
```

Use cases

- Gradually integrate or query another PostgreSQL system.
- Expose selected external data through SQL interfaces.
- Build migration, reporting or federation patterns when justified by the architecture.

8. Geospatial capability for ride-hailing

For a ride-hailing application, location is not just two numbers. The operational question is often: “Which available drivers are near this pickup point, and what is the distance?” PostgreSQL becomes especially powerful when paired with PostGIS.

```
-- Conceptual example using PostGIS
SELECT id,
       ST_DistanceSphere(location,
           ST_MakePoint(:pickup_lng, :pickup_lat)) AS distance_m
FROM drivers
WHERE is_online = true
ORDER BY location <->
    ST_SetSRID(ST_MakePoint(:pickup_lng, :pickup_lat), 4326)
LIMIT 10;
```

The exact functions and index strategy depend on the spatial data type and query pattern. The important architectural point is that the database can perform proximity search with specialized spatial support rather than forcing the application to pull every driver location into memory.

For Jhanvi Cars

- Store driver location in an appropriate spatial representation.
- Index the location column using a spatially suitable index.
- Filter by operational state such as is_online and active vehicle status.
- Then calculate distance / ETA using the appropriate routing service as a separate concern.

9. Advanced SQL: CTEs, recursion, window functions and LATERAL

PostgreSQL is particularly comfortable with complex SQL. MySQL also supports many of these modern SQL features, so the interview distinction is about breadth, maturity and how naturally the features fit complex data models, not about exclusive ownership of SQL constructs.

Window function example

```
SELECT
    driver_id,
    booking_id,
    fare,
    RANK() OVER (
        PARTITION BY driver_id
        ORDER BY fare DESC
    ) AS rank
FROM bookings;
```

This can rank each driver's bookings by fare without collapsing the rows into a GROUP BY result.

Recursive CTE concept

```
WITH RECURSIVE hierarchy AS (
    SELECT id, manager_id, name, 1 AS depth
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    SELECT e.id, e.manager_id, e.name, h.depth + 1
    FROM employees e
    JOIN hierarchy h ON e.manager_id = h.id
)
SELECT * FROM hierarchy;
```

LATERAL

A LATERAL subquery can reference columns from preceding FROM items. It is useful for patterns such as “for each customer, fetch the most recent booking” or “for each driver, select a limited related subset”.

10. Operational visibility: pg_stat_activity and friends

For DevOps and SRE work, advanced database features are only half the story. You also need to understand operational views. pg_stat_activity exposes information about current sessions and backend processes.

```
SELECT
    pid,
    usename,
    datname,
    client_addr,
    state,
    query,
    query_start
FROM pg_stat_activity;
```

Useful questions

- How many sessions are open?
- Which sessions are active versus idle?
- Which queries have been running for a long time?
- Are connections waiting on locks or another resource?
- Is the application exhausting the connection pool or server connection limit?

```
SELECT count(*) AS connections
FROM pg_stat_activity;

SELECT pid, usename, state, query, query_start
FROM pg_stat_activity
WHERE state = 'active'
ORDER BY query_start;
```

11. Putting the features into Jhanvi Cars

Problem	PostgreSQL capability	Possible application use
Driver location	PostGIS + spatial index	Find nearby online drivers
Flexible vehicle metadata	JSONB	Optional vehicle features or external metadata
Booking lookup	B-tree / composite indexes	Fast lookup by customer, driver, status and created_at
Active-only workload	Partial index	Index current bookings without indexing every historical row
Time-window conflicts	Range + exclusion constraint	Prevent overlapping reservations or schedule windows
Admin reporting	Materialized views	Precompute expensive daily/monthly metrics
Data isolation	RLS + roles	Restrict rows by tenant/customer context when required
Operational troubleshooting	pg_stat_activity and related views	Inspect sessions, query state and connection pressure

Architecture reminder

Not every feature belongs in the database. For example, routing/ETA may use an external mapping service, application authentication remains an application concern, and expensive analytics may belong in a separate warehouse. Use database features where they simplify correctness, performance, or data integrity.

12. Senior interview cheat sheet

Question	Strong answer
Why PostgreSQL over MySQL?	Both are mature RDBMSs. PostgreSQL is especially strong when advanced SQL, rich types, extensibility, complex constraints, sophisticated indexes, RLS or geospatial workloads matter.
What is JSONB?	A binary JSON representation optimized for processing, with operators/functions and index support such as GIN for many query patterns.
What is GIN?	An inverted index method useful for data such as JSONB, arrays and text-search-related structures.
What is GiST?	An extensible index framework used for workloads such as ranges and spatial data, depending on operator class.
What is a partial index?	An index defined over a subset of rows that satisfy a predicate, reducing index size and write overhead for targeted workloads.
What is an exclusion constraint?	A database constraint that prevents rows from satisfying conflicting combinations of operators, commonly used to prevent overlapping ranges.
What is RLS?	Row-Level Security applies policies to control which rows a role can read or modify.
What is a materialized view?	A stored query result that can be refreshed, useful for expensive read-heavy reporting.
What is pg_stat_activity?	A system view showing current sessions and backend activity, useful for connection and query troubleshooting.

Golden interview rule

Always connect the feature to a workload. Example: “We used a partial index because 98% of queries target active bookings, and historical rows are rarely searched by that endpoint.” That is stronger than simply listing the feature name.

Official documentation references

PostgreSQL Documentation: https://www.postgresql.org/docs/current/ | Indexes: https://www.postgresql.org/docs/current/indexes.html | JSON types: https://www.postgresql.org/docs/current/datatype-json.html | GIN: https://www.postgresql.org/docs/current/gin.html | GiST: https://www.postgresql.org/docs/current/gist.html | Range types: https://www.postgresql.org/docs/current/rangetypes.html | Constraints: https://www.postgresql.org/docs/current/ddl-constraints.html | Row security: https://www.postgresql.org/docs/current/ddl-rowsecurity.html | Materialized views: https://www.postgresql.org/docs/current/rules-materializedviews.html | Foreign data: https://www.postgresql.org/docs/current/ddl-foreign-data.html | PostgreSQL FDW: https://www.postgresql.org/docs/current/postgres-fdw.html | MySQL Reference Manual: https://dev.mysql.com/doc/refman/8.4/en/